Meeting Transcript Intelligence Platform

Implementation deep-dive and speaking notes (work in progress)

How to use this document
Part 1. The story in two minutes
Part 2. The big picture
  2.1 Why three services
  2.2 Technology
  2.3 Principles that shape the code
Part 3. Data model
Part 4. Authentication and isolation
Part 5. Ingestion in detail
  5.1 Backend side
  5.2 Engine side: `/transcript/process-full`
  5.3 Topic segmentation (Level 2)
  5.4 Meeting intelligence (Level 3)
Part 6. Answering a question in detail
  6.1 Node `extract_filters`: understanding the question
  6.2 Nodes `embed_query` and `vector_retrieval`
  6.3 Node `llm_synthesis`
  6.4 Node `format_citations`
  6.5 Fallback
Part 7. The frontend
Part 8. Achievements, with example queries
  8.1 What the evaluation does and does not show
Part 9. Status: what is finished and what is not
Part 10. Questions your manager may ask
Appendix A. Where things live
Appendix B. Suggested demo order

How to use this document

This is for you to read before you present. Part 1 gives the story to tell. Parts 2 to 7 explain how each piece actually works, in the order data moves through the system. Part 8 covers achievements with example queries, Part 9 lists what is and is not finished (important because the project is still in progress), and Part 10 is a set of questions a technical manager is likely to ask, with honest answers. Every claim here comes from reading the code; where something is unverified or weak, it says so.

Part 1. The story in two minutes

The problem. Teams record meetings, but the knowledge stays buried in long transcripts. Nobody can quickly ask “what did we decide about X?”, “who committed to Y?”, or “which meetings did Z attend?”.

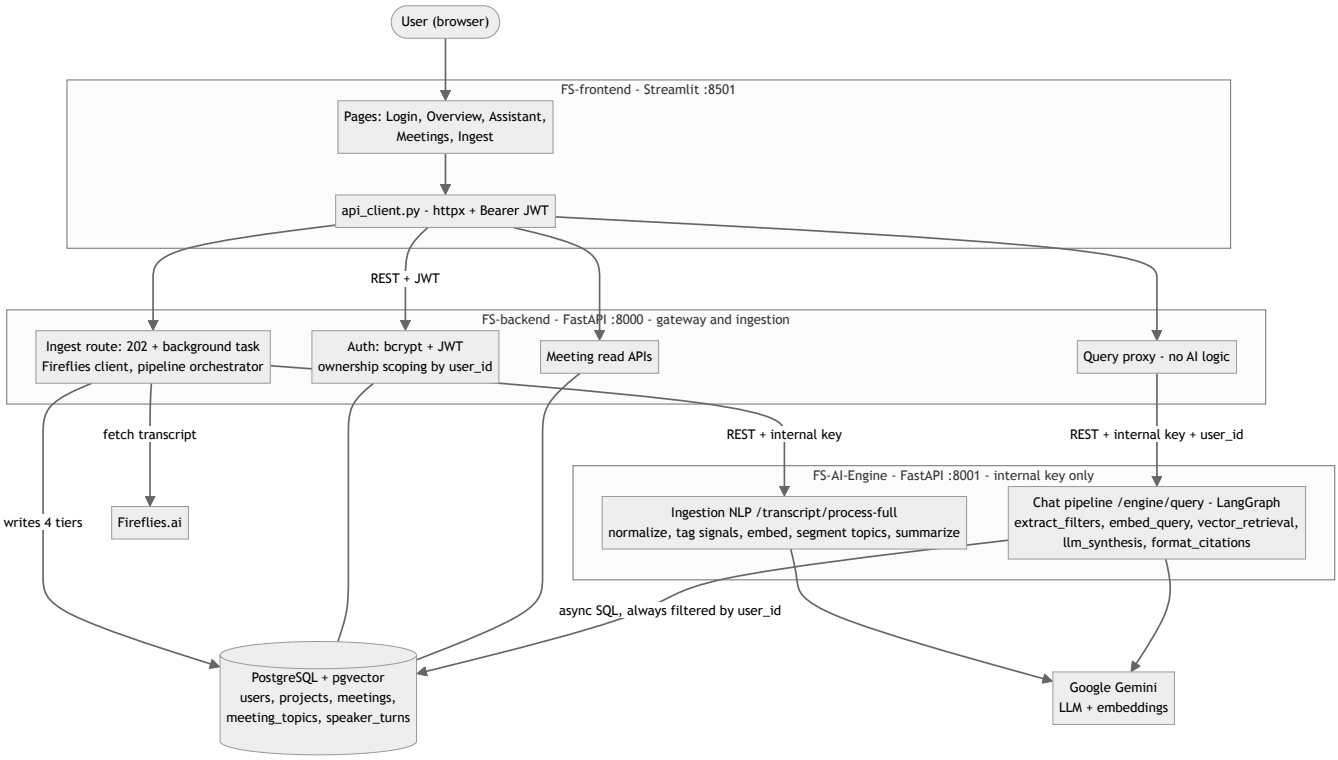
What was built. A three-service platform:

1. **Ingest.** Paste a Fireflies link. The system fetches the transcript, cleans it into speaker turns, tags questions/commitments/blockers, creates vector embeddings, groups turns into topics, drafts a summary, and stores everything in PostgreSQL.
2. **Ask.** A chat interface takes natural-language questions. A LangGraph pipeline works out what was meant (which project, person, meeting, date; is it a database question or a content question), retrieves the relevant turns by vector similarity, and has Gemini write a short, grounded answer.
3. **Trust.** Each answer carries citations to the exact speaker and time. Each user only ever sees their own data.

Where it stands. The core loop works end to end. It is not finished: the richer tiers (topics, summaries) are built but not yet used in answering, retrieval ranking is heuristic, ingestion is link-triggered only, and there is no deployment packaging yet. Part 9 has the full list.

Suggested framing for your manager: “I built the foundation and the core RAG loop with account isolation and an evaluation harness. The next phase is retrieval quality, using the higher tiers, and productionising.”

Part 2. The big picture



2.1 Why three services

Service	Reason it is separate
Frontend (Streamlit)	Fast to build UI. Holds no secrets, no DB, no LLM. Everything goes through one REST client (`services/api_client.py`).
Backend (FastAPI :8000)	The public face: authentication, account scoping, talking to Fireflies, orchestrating ingestion, and writing to the database. It has deliberately **no AI logic**; the query route is a pure proxy.
AI Engine (FastAPI :8001)	All NLP and retrieval. Can be scaled or replaced independently. It trusts nobody except the backend: calls need a shared secret header (`X-Internal-Api-Key`, compared in constant time) and a `user_id`.

2.2 Technology

Layer	Choice
Language	Python 3.11+, `uv` for environments, Ruff for lint
APIs	FastAPI + Pydantic v2
Database	PostgreSQL with the `pgvector` extension

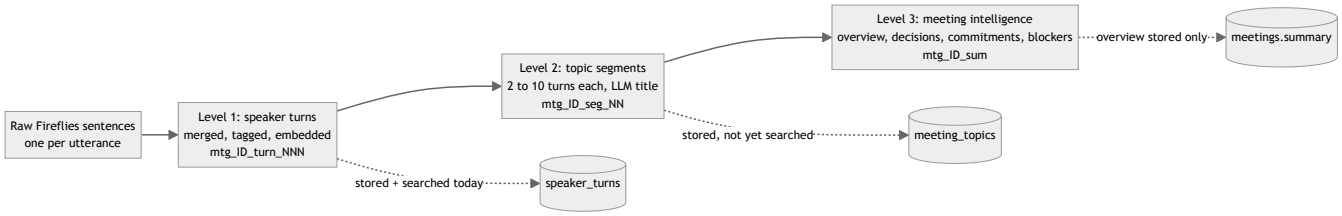
Layer	Choice
ORM	SQLAlchemy sync in the backend, async (asyncpg) in the engine
Migrations	Alembic (revision head stored in a Postgres schema comment rather than an `alembic_version` table)
Orchestration of chat	LangGraph `StateGraph`
LLM and embeddings	Google Gemini through its OpenAI-compatible endpoint, using the `openai` SDK: `gemini-2.5-flash` for text, `gemini-embedding-001` at 1536 dimensions
Transcripts	Fireflies.ai GraphQL
Auth	bcrypt password hashes, HS256 JWT
UI	Streamlit multi-page app
Evaluation	DeepEval golden dataset

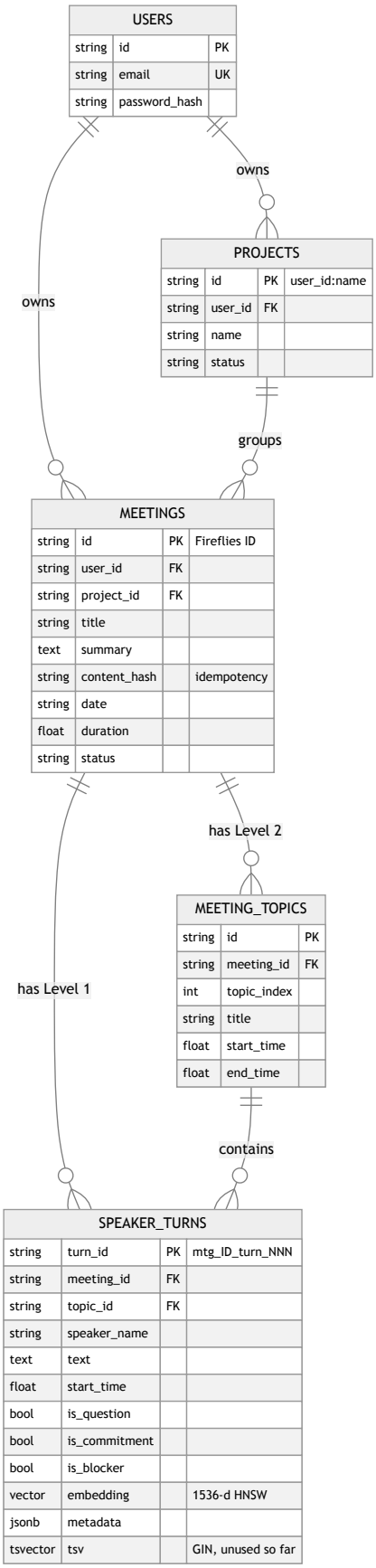
2.3 Principles that shape the code

- **Thin gateway, smart engine.** Easy to explain and to test separately.
- **Isolation by construction.** Every table has `user_id`; the engine's SQL always includes `WHERE user_id = :u_id`. Project IDs are stored as `{user_id}:{name}` so two users can both have a project called "NOLOCODE". Tests assert that naming someone else's meeting, project or speaker returns nothing.
- **Graceful degradation.** If the LLM fails, topic titles and summaries fall back to heuristics. If there is no embedding key, deterministic hash-based vectors are used (for tests and offline development).
- **Deterministic IDs.** `mtg_<id>_turn_<NNN>`, `mtg_<id>_seg_<NN>`, `mtg_<id>_sum` make every item traceable and make citations possible.
- **Let code do the counting.** Meeting counts per project are computed in Python and given to the LLM as facts, because LLMs miscount.

Part 3. Data model

The data is a hierarchy where each level is a coarser view of the one below.





Notes on the `speaker_turns` table, which is where most of the work happens:

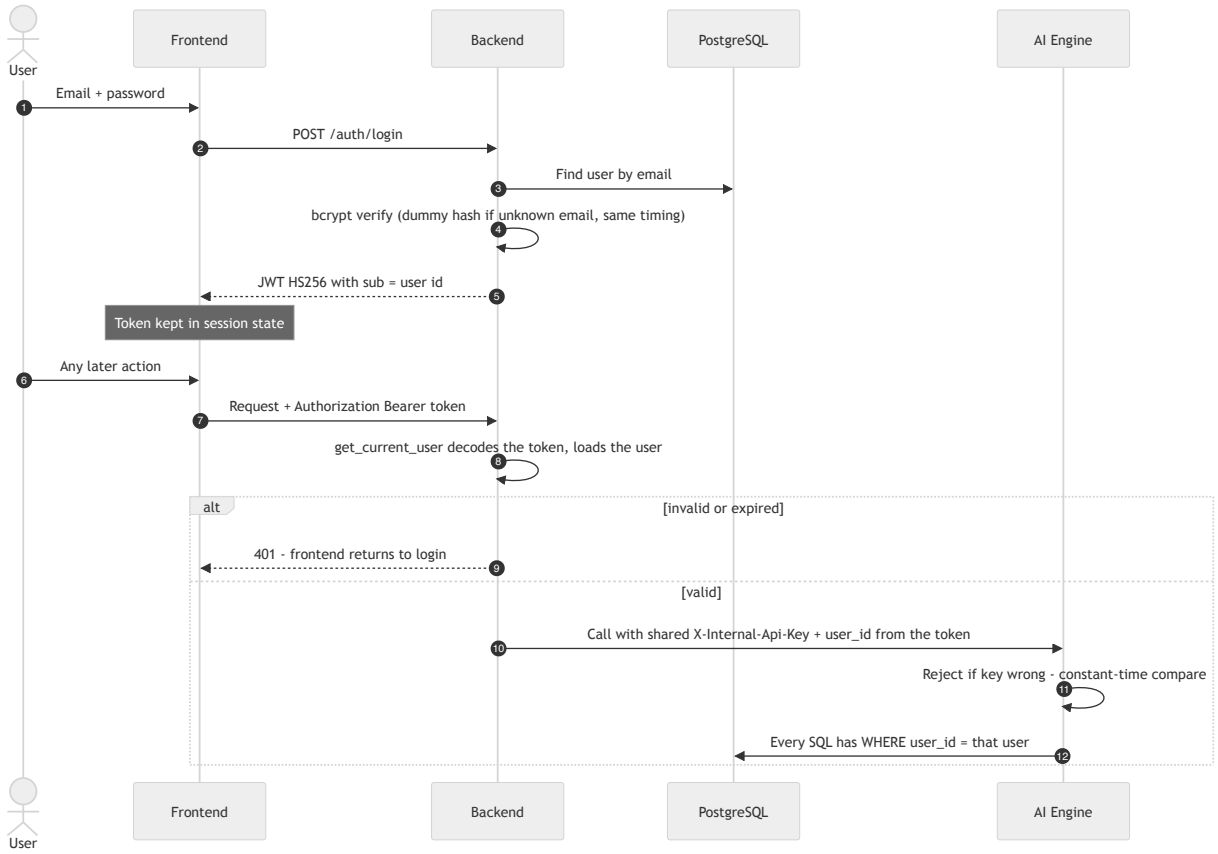
- `embedding vector(1536)` with an **HNSW index** using cosine operators.

- `tsv` is a generated `tsvector` column with a **GIN index** for full-text search. It is created and maintained, but **no query uses it yet**. It is groundwork for hybrid search.
- `metadata` (JSONB) duplicates speaker, timestamp, meeting and project, plus the three signal flags. The engine reads speaker names from here first.
- `is_question`, `is_commitment`, `is_blocker` come from the regex tagger.

`meetings.content_hash` is the SHA-256 used for idempotency. `meetings.date` is stored as the string Fireflies supplies (epoch milliseconds); the engine converts it to `YYYY-MM-DD HH:MM UTC` when needed.

Two Alembic revisions exist: `e0d6e74c6561` (initial schema) and `a3f1c9d27b40` (users and ownership). On startup the backend enables the `vector` extension and runs migrations to head; if it finds a database built by the old `create_all` code, it stamps it at the baseline revision first. If migration fails it refuses to start rather than run on a half-migrated schema.

Part 4. Authentication and isolation

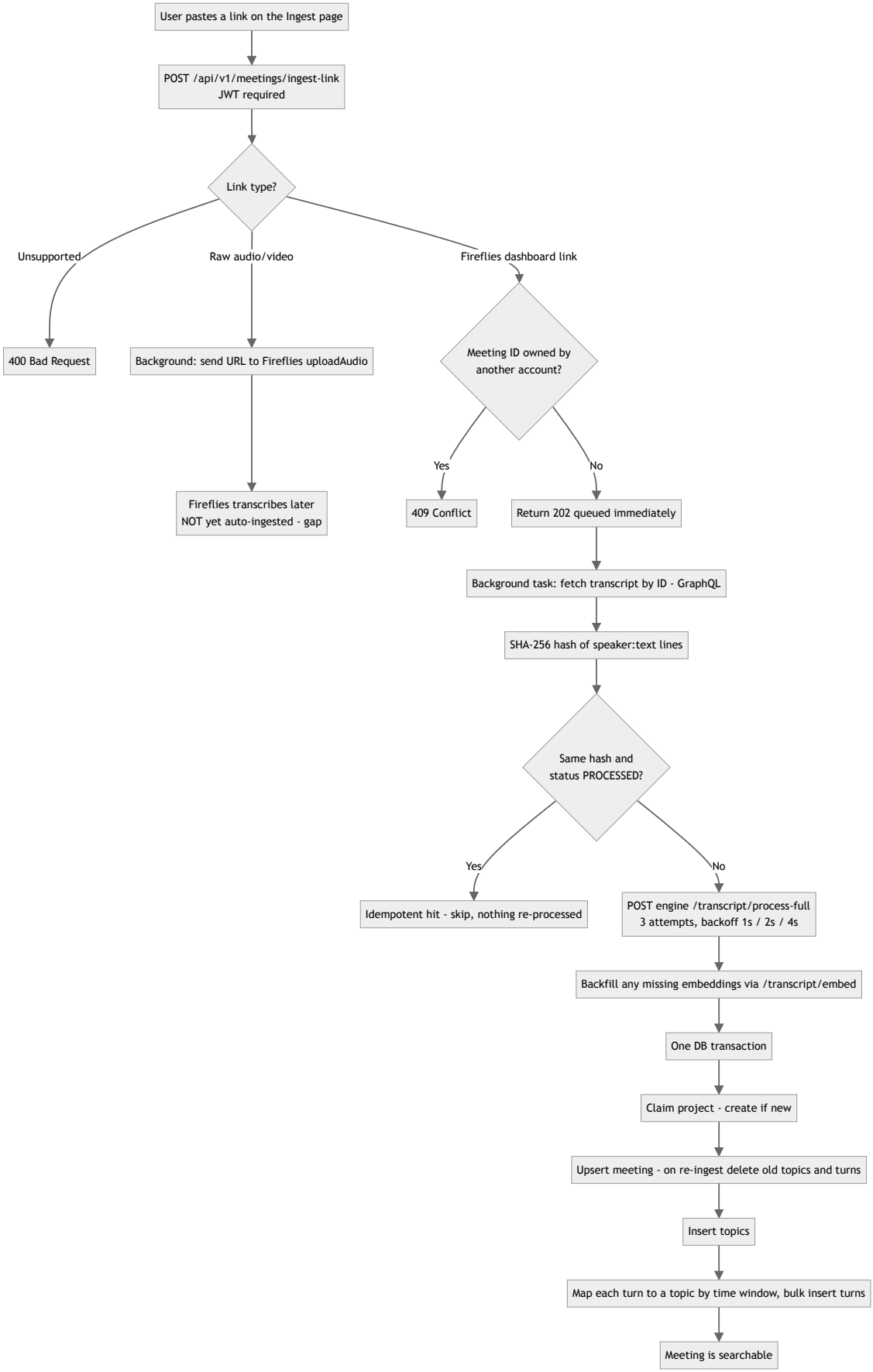


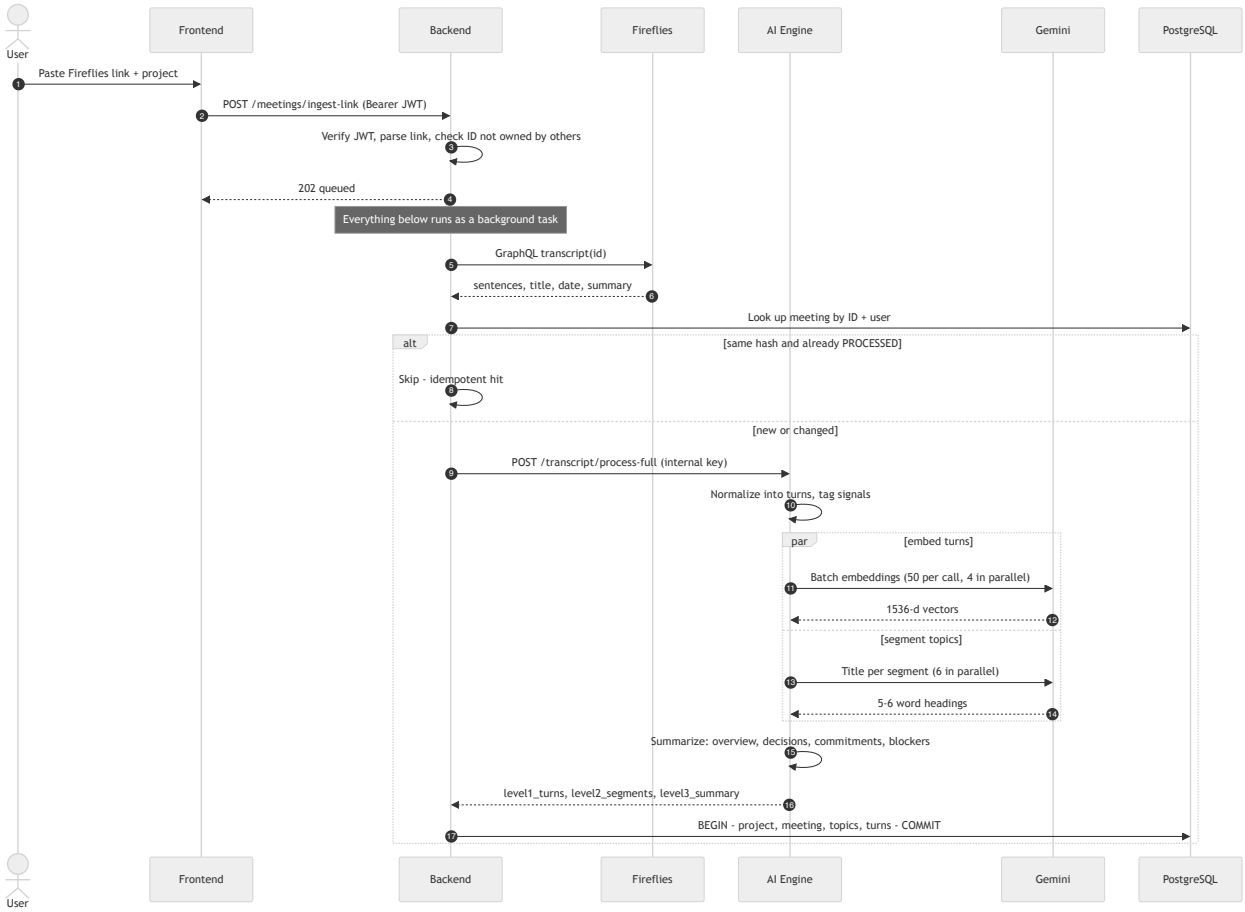
How it works:

1. **Signup/login** (`/api/v1/auth/*`): passwords are hashed with bcrypt (truncated to bcrypt's 72-byte limit). Login verifies against a dummy hash when the email is unknown, so response time does not reveal which emails exist. The response is a JWT whose `sub` is the user ID.
2. **Every other backend route** depends on `get_current_user`, which decodes the token and loads the user. Invalid or expired means 401, and the frontend clears the session and shows the login page.
3. **Backend to engine.** The backend adds the shared internal key and the `user_id` taken from the token, never from the request body. A 401/403 from the engine becomes a 502 at the backend, so the UI does not log the user out for a server misconfiguration.
4. **Meeting ID collisions.** Meeting IDs come from Fireflies and are globally unique primary keys, so ingesting an ID already owned by another account returns 409.

Part 5. Ingestion in detail

5.1 Backend side

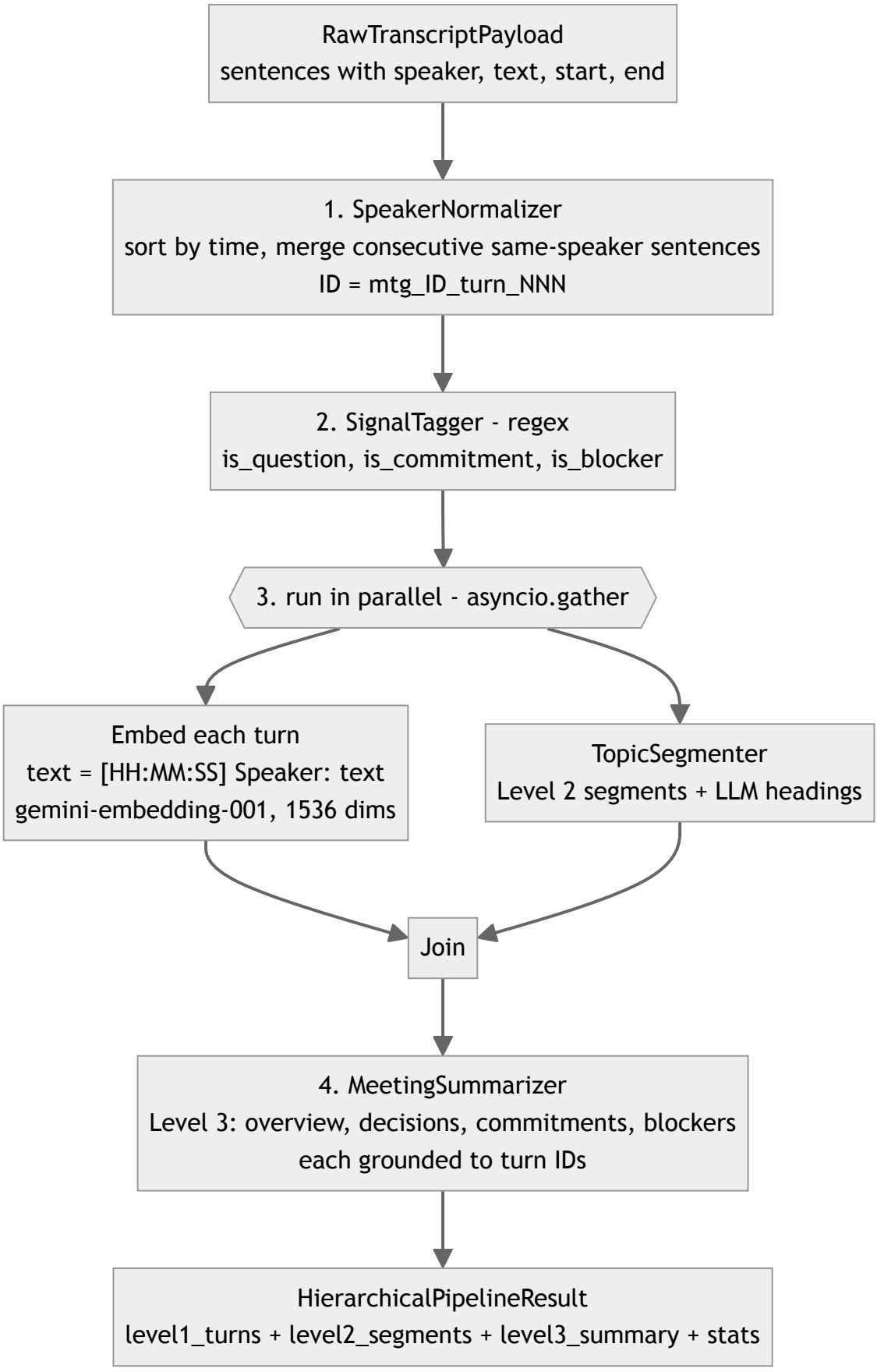

User pastes a link on the Ingest page
POST /api/v1/meetings/ingest-link
JWT required
Link type?
Unsupported
Raw audio/video
Fireflies dashboard link
400 Bad Request
Background: send URL to Fireflies uploadAudio
Fireflies transcribes later
NOT yet auto-ingested - gap
Meeting ID owned by another account?
Yes
No
409 Conflict
Return 202 queued immediately
Background task: fetch transcript by ID - GraphQL
SHA-256 hash of speaker:text lines
Same hash and status PROCESSED?
Yes
No
Idempotent hit - skip, nothing re-processed
POST engine /transcript/process-full
3 attempts, backoff 1s / 2s / 4s
Backfill any missing embeddings via /transcript/embed
One DB transaction
Claim project - create if new
Upsert meeting - on re-ingest delete old topics and turns
Insert topics
Map each turn to a topic by time window, bulk insert turns
Meeting is searchable



Key points to explain:

- **Link parsing.** Fireflies URLs of the form `.../view/<title>::<id>` or `/view/<id>` yield the transcript ID and a title taken from the slug. Direct audio/video links (`.mp3 .mp4 .wav .m4a ...`) are recognised as raw media.
- **Asynchronous by design.** The route validates and returns **HTTP 202** immediately; a FastAPI `BackgroundTask` does the slow work. Consequence: the user gets no completion signal, and failures are only logged (a known gap).
- **Idempotency.** The hash covers `speaker:text` lines. Same hash plus status PROCESSED means skip. If the transcript changed, the meeting is updated and its old topics and turns are deleted and rebuilt inside the same transaction.
- **Resilience.** The call to the engine has a 300 s read timeout and up to 3 attempts with 1 s, 2 s, 4 s backoff on 5xx or network errors. A 4xx fails immediately.
- **Atomic write.** One transaction: claim project (create if new), upsert meeting, insert topics, then insert turns in bulk. Each turn is attached to a topic by checking which segment's time window contains its start time. Any exception rolls the whole thing back.
- **Summary choice.** The meeting's stored summary prefers Fireflies' own overview and falls back to the engine's generated overview.
- **Raw media.** Only forwarded to Fireflies' `uploadAudio` mutation. The resulting transcript is **not** pulled back automatically yet.

5.2 Engine side: `/transcript/process-full`



Step 1, Normalizer. Fireflies gives one record per sentence. The normalizer sorts by start time, drops empty text, and merges consecutive sentences by the same speaker (same speaker ID or same name, case-insensitive) into one **Level 1 turn** with start/end/duration and an ID like `mtg_01KTX..._turn_007`. This is the unit that gets embedded, searched and cited.

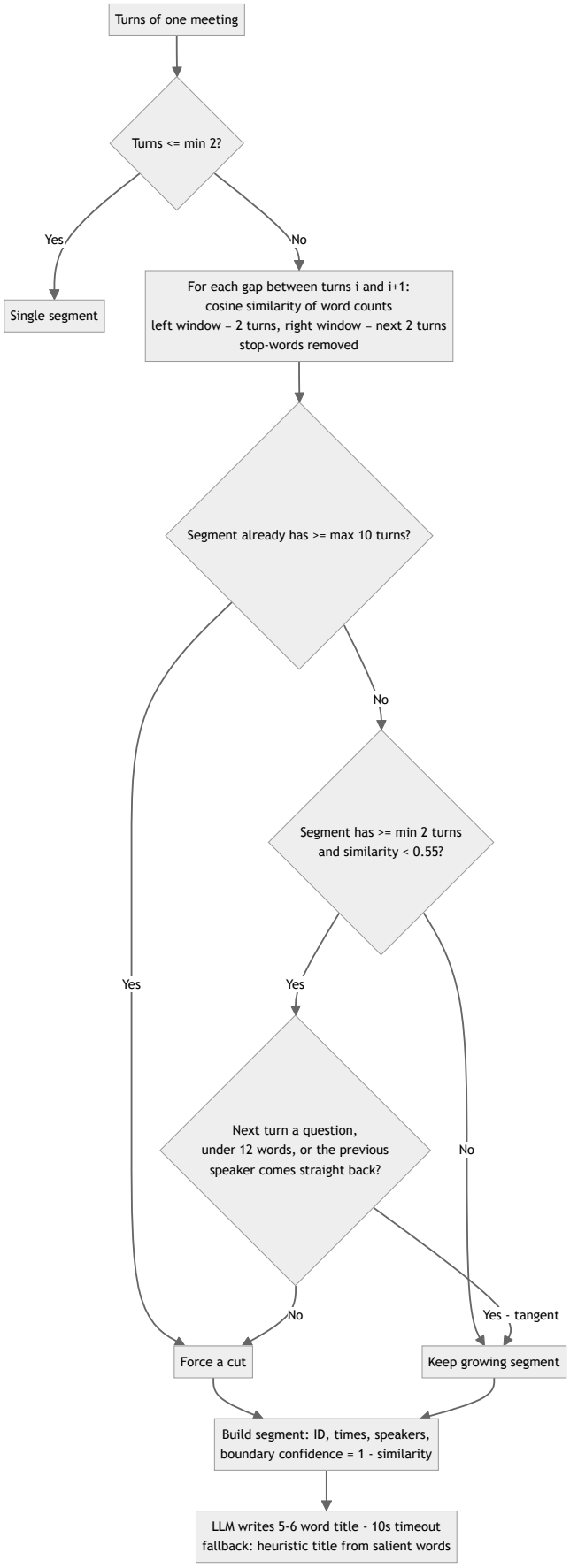
Step 2, Signal tagger. Regex lists mark each turn as a question (“what/why/how”, trailing “?”), a commitment (“I will”, “I’ll follow up”, “by Friday”), or a blocker (“blocked by”, “waiting on”, “stuck on”). It is cheap and deterministic. Limitation: it is lexical, so it misses paraphrases and over-triggers on words like “what”.

Step 3, Embeddings and segmentation in parallel.

- *Embeddings.* Each turn becomes `[HH:MM:SS] Speaker: text`, embedded with `gemini-embedding-001` at 1536 dimensions, in batches of 50 with 4 concurrent calls, retrying without the `dimensions` argument if the endpoint rejects it. Vectors are padded or truncated to 1536 if needed. Putting speaker and time into the embedded text lets “what did Neha say” match on the name as well as the content.
- *Segmentation* (next section).

Step 4, Summarizer. See 5.4.

5.3 Topic segmentation (Level 2)

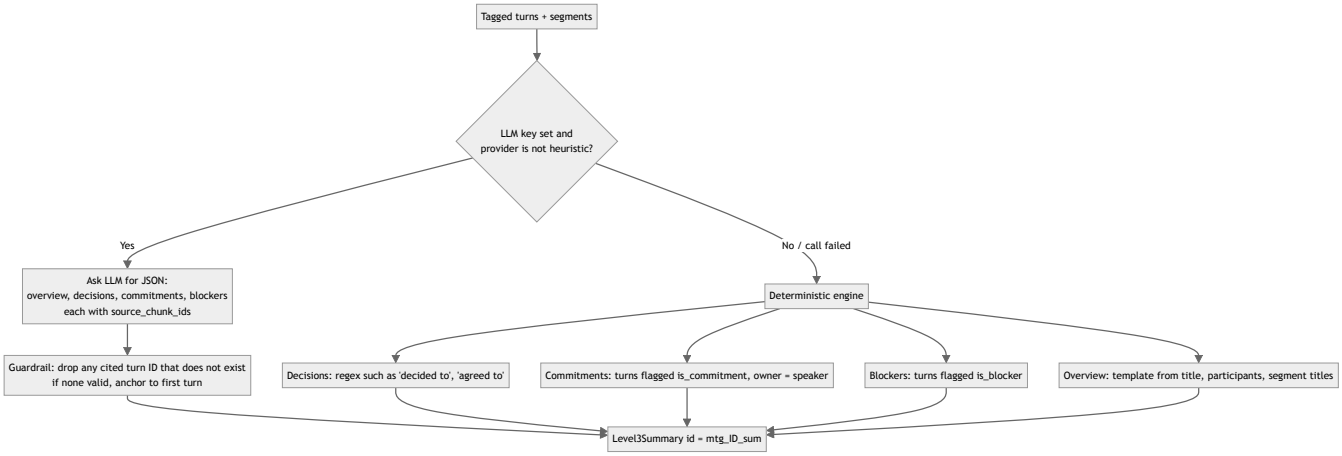


This is a **lexical** algorithm, not an embedding-based one. For each boundary between turns it compares the word counts of the two turns before it with the two turns after it (cosine similarity, stop-words removed). A low score suggests a topic change. Guardrails stop it from over-splitting: minimum 2 and maximum 10 turns per segment, threshold 0.55, and a “tangent” rule so a short question, a reply under 12 words, or the

previous speaker immediately coming back does not start a new topic. Each segment gets a boundary confidence (one minus the similarity) and an LLM-written 5 to 6 word heading, six generated concurrently with a 10 s timeout and a heuristic fallback.

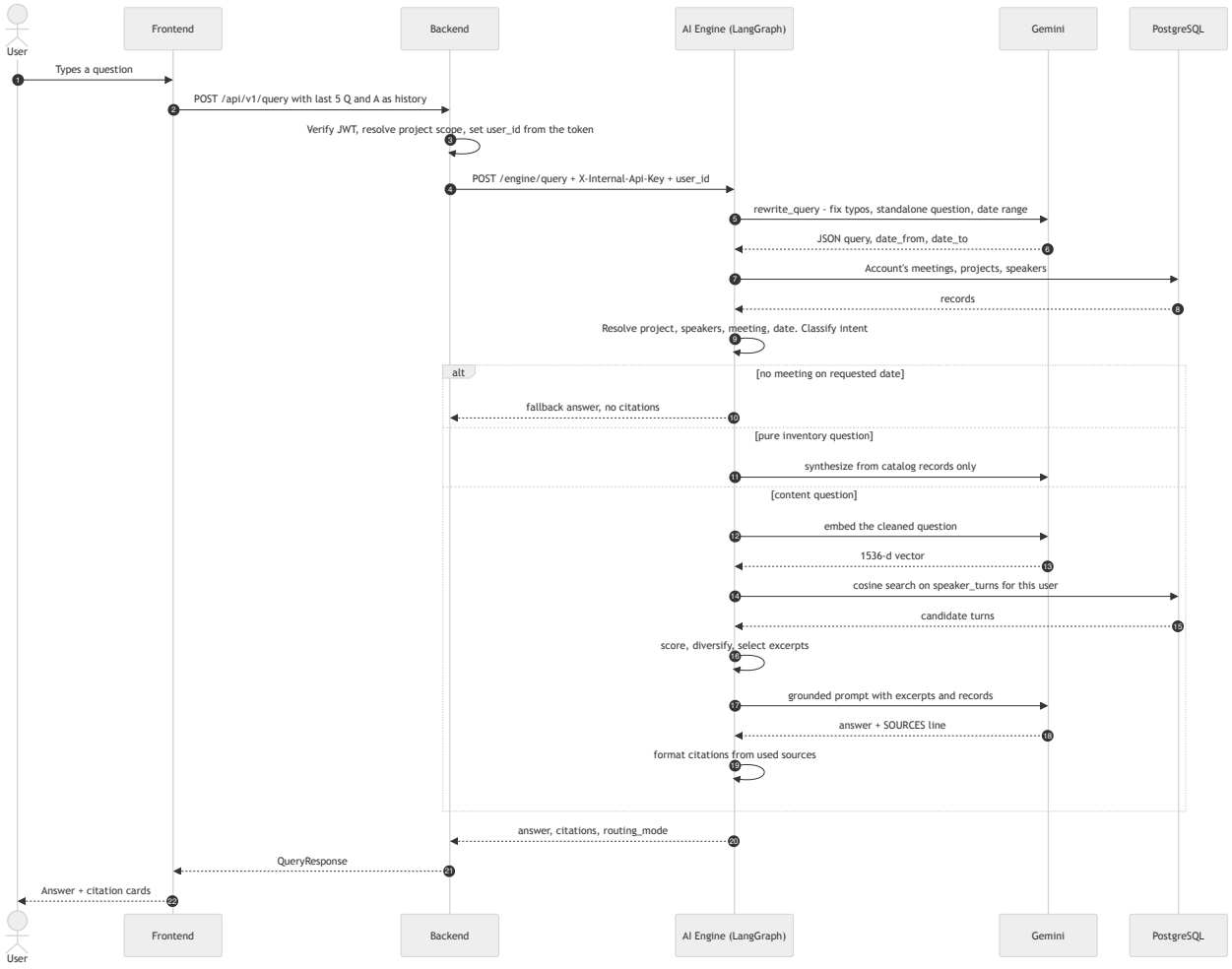
Honest caveat: using turn embeddings (already computed) for the similarity would likely be more accurate. This was the simpler first version.

5.4 Meeting intelligence (Level 3)

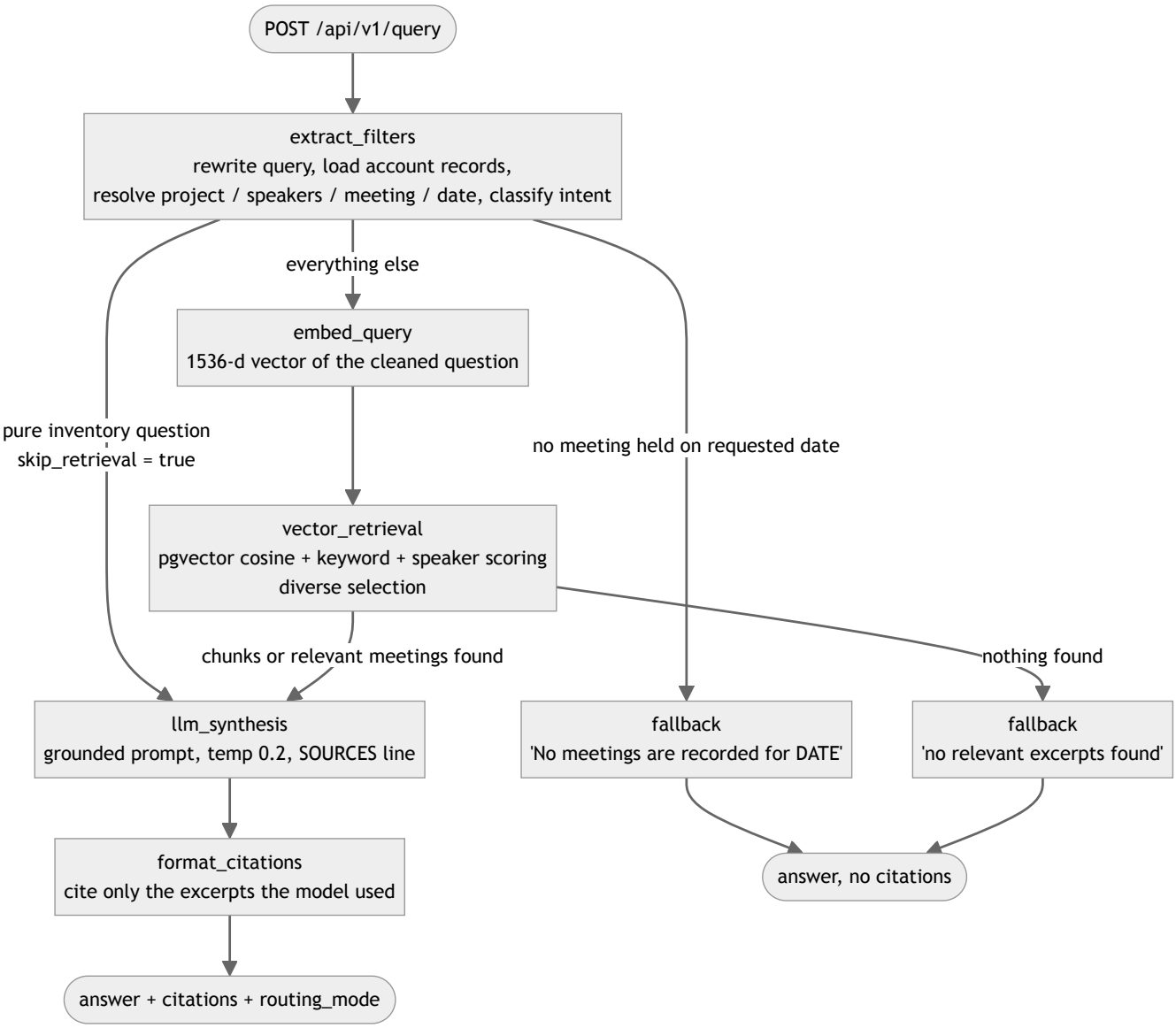


The summary node produces an overview and lists of decisions, commitments (with owner) and blockers, **each tied to the turn IDs it came from**. If an LLM is used, a guardrail removes any turn ID it invented. Honest caveats: (a) the LLM branch only runs when a separate `llm_api_key` is configured and calls the OpenAI endpoint directly, unlike the rest of the system which uses Gemini, so in practice the deterministic regex branch usually runs; (b) the backend currently persists **only the overview text**. Decisions, commitments and blockers are computed and then discarded. Also, the topic summary text is not saved because the backend reads a `summary` / `compiled_text` key while the engine returns `topic_summary`.

Part 6. Answering a question in detail

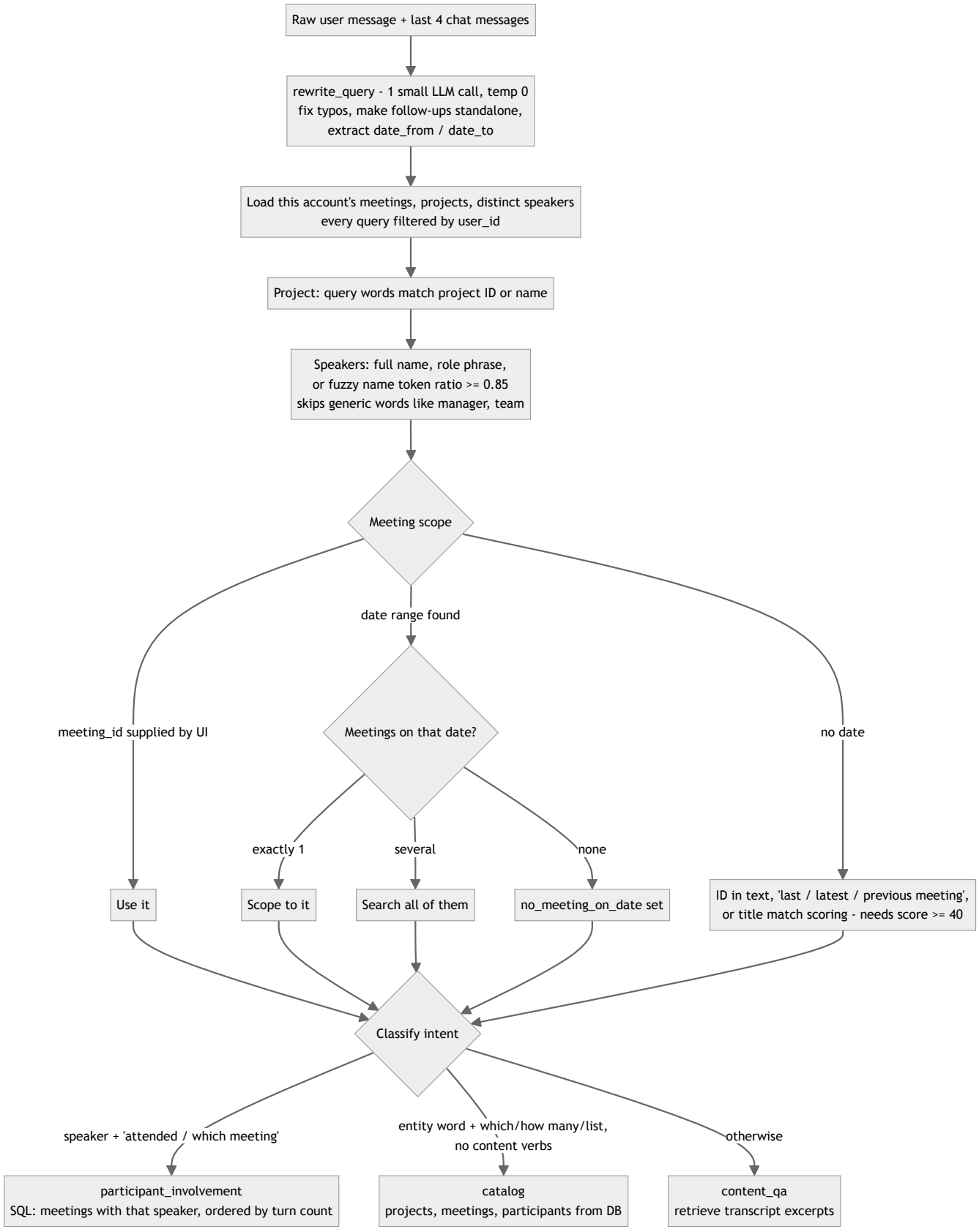


The frontend sends the question plus the last five question-and-answer pairs. The backend resolves the project scope, attaches the user ID from the token, and proxies to `/api/v1/engine/query`. Everything else happens in a compiled LangGraph.



The shared `ChatState` carries everything between nodes: query, user, scope, intent, retrieved chunks, answer, citations, routing mode and latency. The graph has three exits: the normal path, a catalog shortcut that skips embeddings and vector search, and a fallback.

6.1 Node `extract_filters`: understanding the question



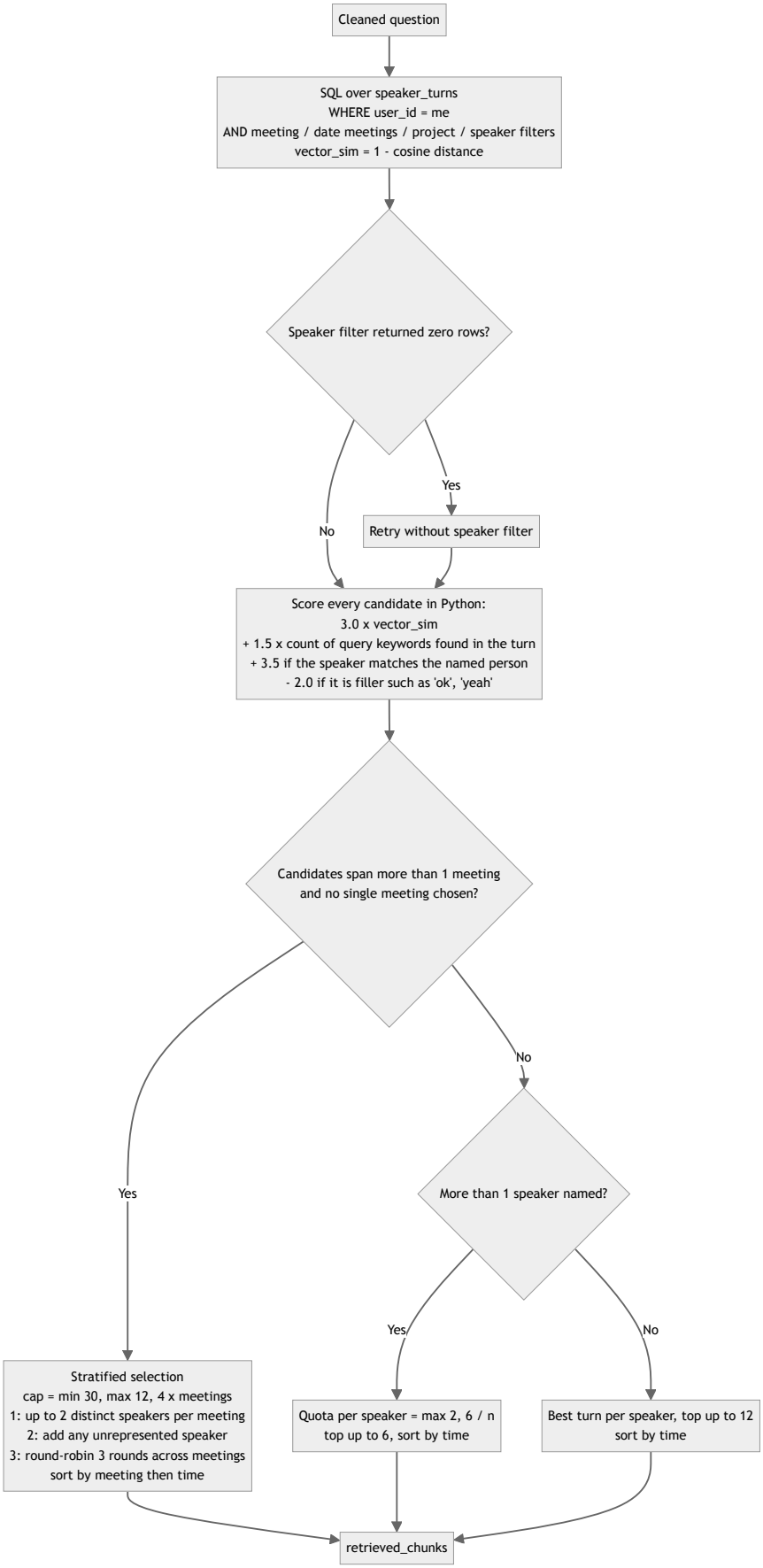
This is the most intricate node. In order:

1. **Rewrite (one small LLM call, temperature 0).** Fixes typos using the whole sentence (“rate of the meeting” becomes “date of the meeting”), makes follow-ups standalone using the last four messages, and extracts a date range of when the *meeting* took place (relative words are computed from today’s date). If this call fails, the original query is used with no dates.

2. **Load the account's records**: all its meetings (ID, title, project, date), projects, and distinct speaker names. Always filtered by `user_id`.
3. **Project**: if the query contains a known project ID or name, scope to it.
4. **Speakers** (three strategies): full-name match; two-word role or name phrase; fuzzy single-name token (difflib ratio at least 0.85). A stop list of generic words ("manager", "team", "support"…) and project names prevents "the Nolocode project" from being mistaken for a person named Nolocode.
5. **Meeting scope**, in priority order: ID passed by the UI; a date range from step 1 (one match scopes to it, several are searched together, none sets `no_meeting_on_date`); a meeting ID in the text; "last / latest / previous meeting" (newest by date); title match scoring, requiring a strong match (at least 40 points, no single stray keyword). Phrases like "across all meetings" deliberately avoid locking to one.
6. **Intent**:
    - *participant_involvement*: a speaker was found and the question uses words like "attended" or "which meeting". The node runs SQL to list that person's meetings by turn count.
    - *catalog*: the question names an entity (project, meeting, participant, topic), uses a listing/counting word, has no content verbs ("said", "decided") and no speaker. Typo correction (one edit away, words of 5+ letters) is applied first, so "meeitngs" still works.
    - *content_qa*: everything else.
7. **Skip retrieval?** Only when every word in the question is inventory vocabulary or a known project token. "How many meetings are in NOLOCODE" skips retrieval; "what decisions in NOLOCODE" does not. A wrong guess here costs speed, never correctness.

6.2 Nodes `embed_query` and `vector_retrieval`

The cleaned question is embedded with the same model. Retrieval then runs one SQL statement against `speaker_turns` joined to meetings and topics, filtered by user, meeting (or the set of meetings on a date), project and speaker, and returns each turn with `vector_sim = 1 − cosine_distance`. If a speaker filter returns nothing, it retries without it.



The score blends signals: 3 times vector similarity, 1.5 times the number of query keywords found in the turn, a 3.5 bonus if the speaker is the person named, and a 2 penalty for filler such as “ok” or “yeah”. Selection is then tailored to the situation. Across several meetings it deliberately **spreads** the picks (two

speakers per meeting, then any missing speakers, then a round-robin) so a cross-meeting answer isn't dominated by one meeting. This is what makes "across all meetings" questions work.

Honest caveats to be ready for:

- The SQL has **no LIMIT**. All in-scope turns are fetched and scored in Python. It works at the current data size (hundreds to low thousands of turns) but does not scale, and the HNSW index is not being exploited. Fix: top-k in SQL with a candidate pool, then rerank.
- The scoring weights are hand-tuned, not learned.
- The full-text index is unused; the keyword component is a simple substring count.

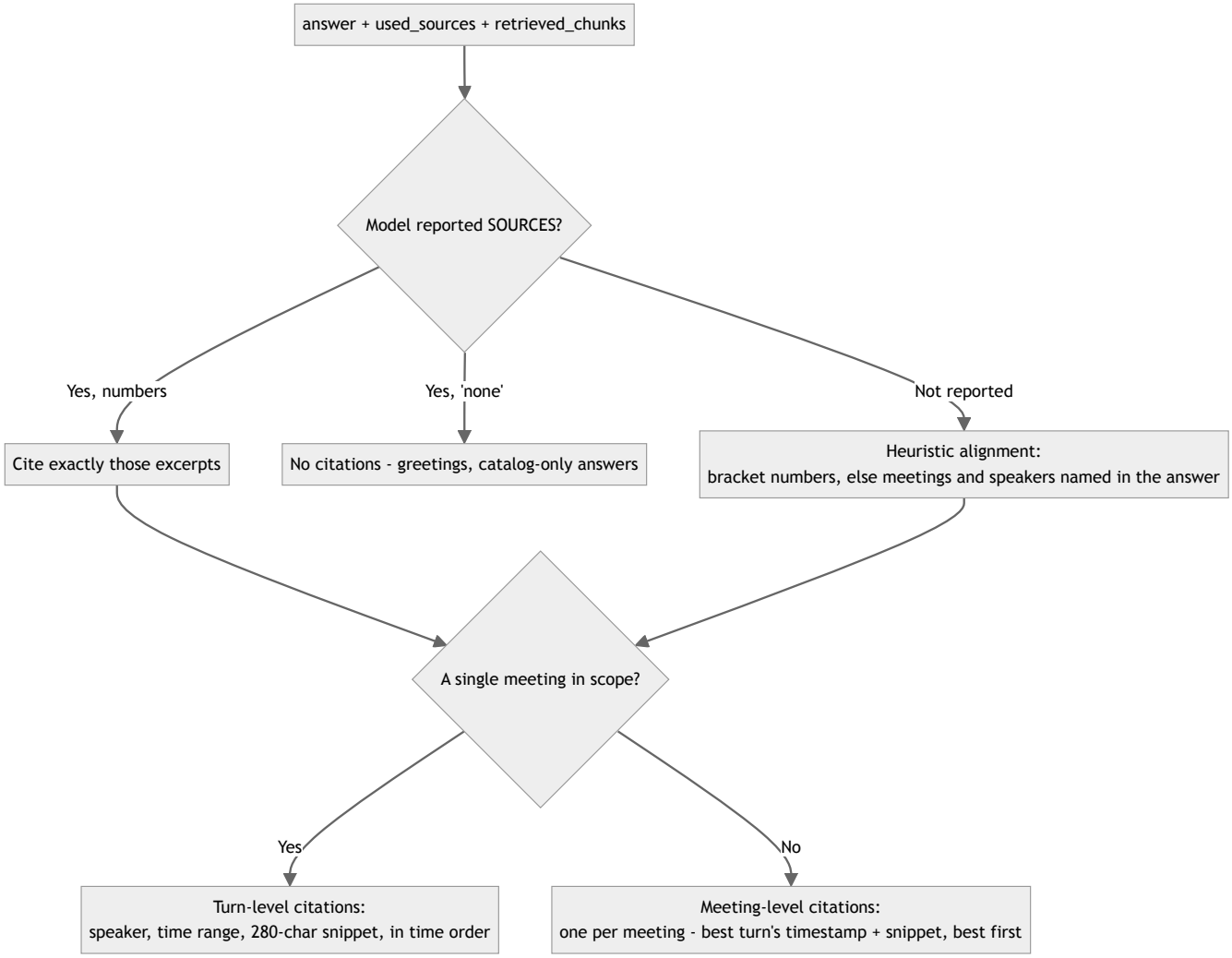
6.3 Node `llm_synthesis`



The prompt contains: numbered excerpts (each with meeting title, **held-on date**, topic, speaker, timestamp, quote); catalog records when relevant; per-meeting details; involvement records; prior chat turns; and a system prompt that sets the tone and rules. The most important rules: answer in 2 to 5 sentences unless asked for more; never print meeting IDs, give title and date; treat today's date as the anchor for "yesterday" and never confuse it with dates *spoken inside* a transcript; use only the supplied context and say so when information is missing; finish with a line `SOURCES: 2, 5` naming the excerpts relied on (or `none`). The reply is parsed to strip that line and any inline `[3]` markers.

Each content question therefore costs: one rewrite LLM call, one embedding call, one synthesis LLM call (a catalog-only question skips the embedding; a fallback skips synthesis).

6.4 Node `format_citations`



Because the model states which excerpts it used, citations show exactly those, not “everything retrieved”. Greetings and catalog answers show none. If a single meeting is in scope you get turn-level cards with a 280-character snippet; otherwise one card per meeting, best first. If the model gives no SOURCES line, a heuristic matches meeting titles, speaker names and `[n]` markers in the answer.

6.5 Fallback

Two cases: no meeting held on the requested date (“No meetings are recorded for 2026-06-13”) and nothing retrieved. Both return without calling the synthesis LLM, so the system never makes up an answer when it has nothing.

Part 7. The frontend

`app.py` configures Streamlit, shows the login page until authenticated, then registers four pages with `st.navigation`:

Page	Purpose
Overview	KPIs, project breakdown, recent meetings, and a quick-ask box that hands off to the Assistant
Assistant	Chat. A shared “scope” panel lets the user pin a project, meeting or speaker. It builds history from the last five questions and renders answers with citation cards
Meetings	Per-meeting tabs for summary, speaker turns, and speaker statistics
Ingest	Form for the link, optional project and title; calls `/ingest-link` and reports “queued”

`services/api_client.py` wraps all calls, adds the bearer token, extracts clean error messages, and flags the session as expired on any 401.

Part 8. Achievements, with example queries

The example data is a real project (NOLOCODE) of eight meetings from April to October 2026. “Verified” means the answer was read in the 2026-10-08 benchmark run.

#	Achievement	Example query	Outcome
1	Per-user isolation	Second account asks *“How many meetings are there?”*	Sees only its own. Automated tests cover retrieval, catalog, involvement and cross-account naming.
2	Idempotent ingestion	Submit the same link twice	Second run skipped by content hash
3	Database-grounded catalog answers	*“How many meetings are there under the NOLOCODE project?”*, *“list all the participants”*	Exact counts and names; no vector search
4	Typo tolerance	*“how many meeitngs are there?”*	Understood as meetings
5	Participant involvement	*“Which meetings did Karan Middha attend?”*	Meetings with titles and dates
6	Factual retrieval with speaker	*“What AWS EC2 instance type did parteek chaurasiya specify in the M4 and DevOps discussion?”*	m6g.large (verified)
7	Commitments	*“What did Neha commit to doing before finalizing the PDF report generation in the Nolocode meeting?”*	Share screen and PDF first (verified)
8	Meeting scoping by ID/title	*“In meeting 01KQBRC3M5TCY9WDC7SQVV3X6F, how many charts did Neha say must be shown for Cash Flow and Runway?”*	Three (verified)
9	Relative time	*“What happened in the latest call?”*	Resolves to the newest meeting, 5 Oct 2026 (unit-tested)
10	Dates	*“What was discussed on June 12?”* then *“…June 13?”*	Two meetings searched together; then “No meetings are recorded for 2026-06-13”
11	Follow-ups	*“Who attended the S3 meeting?”* then *“What did he say about buckets?”*	Pronoun resolved from history
12	Cross-meeting synthesis	*“Across all meetings, what AWS services were discussed?”*	S3, EC2, ECS, ECR, Redis, RabbitMQ
13	Refusing to guess	*“What discount did we negotiate for the Snowflake subscription?”*	Says records do not contain it

#	Achievement	Example query	Outcome
14	Citations	Any content question	Cards with speaker, time and snippet, only for excerpts used
15	Benchmark harness	`run_deepeval_evaluation.py`	50 cases across 10 scenario types

8.1 What the evaluation does and does not show

The latest run (2026-10-08): 50 queries, 49 returned an answer, average latency 6.4 s (this includes two LLM calls plus embedding). **That 98% is availability, not accuracy.** No correctness score has been computed; the dataset stores expected and actual answers side by side for DeepEval metrics, which still need to be run. Reading the output, most factual, summary and negative-probe answers look right, and some are wrong:

- “When did Ngũmi Gituro commit to sending the 13-week forecast sheet?” was answered as never committed.
- “What trade-off … did Ashpreet Singh and Karan Middha discuss regarding S3?” was answered as never discussed, although the expected answer says it was.

These are retrieval misses, consistent with the limits in 6.2. The older 20-case runner shows 0% passed only because it predates authentication (all calls get 401); it needs a login step.

Say this proactively. It shows you know where the system is weak.

Part 9. Status: what is finished and what is not



Area	Status	Notes
Auth, isolation	Done	Tested
Link ingestion, persistence, idempotency	Done	Only Fireflies dashboard links
Chat pipeline (catalog, involvement, content, dates, history, citations)	Done	Engine unit tests exist
Topics (L2) and summaries (L3)	Partial	Computed, but retrieval ignores them; decisions/commitments/blockers and topic summary text are not saved
Retrieval quality	Partial	Heuristic scoring, no reranker, no SQL top-k, unused full-text index
Raw audio and webhooks	Not started	Audio is forwarded to Fireflies only; no webhook; no job status
Packaging	Not started	Dockerfiles per service, no compose
Tests	Partial	None for backend or frontend
Evaluation scoring	Partial	Harness runs; correctness metrics not yet computed
Clean-up	Open	Legacy OpenRouter naming in config; summarizer calls OpenAI endpoint; CORS `*`; duplicated schemas

Proposed next phase (in priority order):

1. Run DeepEval correctness metrics to get a real baseline.
2. Retrieval: top-k in SQL, hybrid vector + full-text, add a reranker; re-measure.
3. Use topics and summaries for broad questions (“summarise this meeting” should read Level 2/3, not scatter across turns). Persist decisions, commitments and blockers so they can be queried directly (“list open action items”).
4. Webhook-driven ingestion with job status and visible errors; auto-ingest uploaded audio.
5. docker-compose, backend and frontend tests, secrets and CORS hardening.

Part 10. Questions your manager may ask

Why RAG rather than putting whole transcripts in the prompt? Transcripts across many meetings exceed context limits and cost; retrieval picks the few relevant turns and enables citations.

Why a graph (LangGraph) and not one function? The flow has real branches (catalog shortcut, no-meeting-on-date, no-results fallback). A graph makes those explicit, testable and extensible, for example by adding a reranker node.

Why does the backend not just call Gemini? Separation of concerns: the backend authenticates and owns persistence; the engine owns AI and can change models or scale on its own.

How do you stop one user seeing another's data? Identity comes from a signed JWT; the backend injects `user_id`; every engine query filters on it; project IDs are namespaced; meeting IDs are claimed globally. Tests try to cross the boundary.

How do you reduce hallucination? Strict-grounding system prompt, only retrieved excerpts and database facts in context, counts computed in code, a fallback that never calls the LLM when nothing matches, and citations limited to what the model reports using. It is reduced, not eliminated; correctness metrics are the next step.

Why is a query about 6 seconds? Two sequential LLM calls (rewrite, then synthesis) plus an embedding call and an unbounded vector scan. Options: skip the rewrite for clean queries, top-k in SQL, stream the answer.

Does it scale? Ingestion is fine (batched, parallel, idempotent). Retrieval does not yet, because of the missing SQL limit. That is the first fix.

What is the weakest part? Be direct: retrieval ranking and the unused higher tiers. The evaluation shows a few factual misses that come from it.

What if Fireflies or Gemini is down? Ingestion retries the engine three times with backoff; engine falls back to heuristics for titles, summaries and (offline) embeddings. Query rewrite failure keeps the original query. Synthesis failure returns an error (no secondary model yet).

What would you do with more time? The list in Part 9.

Appendix A. Where things live

Concern	File
Chat graph topology and routing	`FS-AI-Engine/engine/graphs/chat_graph.py`
Filter extraction node	`engine/graphs/chat_nodes.py`
Intent, speaker and meeting resolution, typo fix	`engine/graphs/chat_query.py`
Retrieval and scoring	`engine/graphs/chat_retrieval.py`
Prompt, rewrite, synthesis, fallback	`engine/graphs/chat_synthesis.py`
Catalog and meeting-detail records	`engine/graphs/chat_records.py`
Citations	`engine/graphs/chat_citations.py`
Ingestion endpoints	`engine/api/routes.py`
Normalizer, tagger, segmenter, summarizer, embeddings	`engine/services/*.py`
Ingestion orchestrator	`FS-backend/app/services/pipeline_orchestrator.py`
Link route, query proxy	`FS-backend/app/api/routers/meetings.py`, `query.py`
Auth	`FS-backend/app/api/v1/endpoints/auth.py`, `core/security.py`
Ownership rules	`FS-backend/app/services/ownership.py`
Schema and migrations	`FS-backend/app/core/database.py`, `alembic/versions/`
Chat UI	`FS-frontend/app_pages/assistant.py`
Benchmark	`evaluations/`

Appendix B. Suggested demo order

1. Log in; show Overview.
2. Ingest page: paste a link; explain 202 and background work.
3. Meetings page: show turns and topics created.
4. Assistant: *“How many meetings are there under NOLOCODE?”* (catalog, no retrieval).
5. *“What AWS EC2 instance type did parteek chaurasiya specify in the M4 and DevOps discussion?”* (factual, citation).
6. *“What happened in the latest call?”* then a follow-up (time and history).
7. *“What was discussed on June 13?”* (graceful no-result).
8. *“What discount did we negotiate for Snowflake?”* (refuses to guess).
9. Show the architecture diagram and the status slide.